

Chapter 5 Amortized Analysis

Amortized Analysis of Dynamic Arrays

◆ PROMPT 1: What is a dynamic array?

✓ Answer:

A **dynamic array** is an array that:

- Stores elements in **contiguous memory**
- Automatically **resizes itself** when it becomes full

Examples:

- Python list
- Java ArrayList
- C++ vector

◆ PROMPT 2: What operation are we analyzing?

✓ Answer:

We analyze the **append (push-back)** operation:

```
append(x)
```

which adds an element at the end of the array.

◆ PROMPT 3: Why does append sometimes take $\Omega(n)$ time?

✓ Answer:

Most appends are fast, but **sometimes**:

- The array is full
- A **new array of double size** is created
- All existing elements are **copied**

If there are n elements:

copy cost = $\Omega(n)$

So a **single append** can be expensive.

◆ PROMPT 4: Then why do we say append is $O(1)$ amortized?

✓ **Answer:**

Because:

- Expensive operations happen **rarely**
- Cheap operations happen **very often**
- When averaged over many operations, cost per append is constant

This is called **amortized analysis**.

◆ PROMPT 5: What is amortized analysis?

✓ **Answer:**

Amortized analysis studies:

The **average cost per operation over a sequence of operations**

Not probability-based.

Not average-case input.

It's a **guarantee over time**.

◆ PROMPT 6: What strategy does the dynamic array use?

✓ **Answer:**

Doubling strategy:

- If array capacity = k
- When full $\rightarrow$ create new array of size $2k$
- Copy k elements

◆ PROMPT 7: Why is doubling important?

✓ Answer:

Doubling ensures:

- Fewer resizes
- Total copying work stays linear

If we increased by +1 each time $\rightarrow O(n^2)$ ✗

If we doubled $\rightarrow O(n)$ ✓

◆ PROMPT 8: What is the accounting (cyber-dollar) method?

✓ Answer:

Think of the computer as:

A coin-operated machine 💰

- Each basic operation costs **1 cyber-dollar**
- We can **overcharge** some operations
- Extra money is saved to pay for future expensive operations

◆ PROMPT 9: What assumptions do we make?

✓ Answer:

1. Normal append (no resize) costs **1 dollar**
2. Resizing from size k to $2k$ costs **k dollars**
3. Each append is charged **3 dollars**

◆ PROMPT 10: Why charge 3 dollars for append?

✓ Answer:

- 1 dollar $\rightarrow$ pays for the append itself
- 2 dollars $\rightarrow$ saved for future resizing

This creates a **bank balance**.

◆ PROMPT 11: Where are the extra cyber-dollars stored?

✓ Answer:

They are conceptually **stored with the array cells**.

Each element inserted carries **2 saved dollars**.

◆ PROMPT 12: When does overflow happen?

✓ Answer:

Overflow happens when:

`number of elements = capacity =  $2^i$`

Example:

`capacity = 8`

`elements = 8 → overflow on next append`

◆ PROMPT 13: How do we pay for resizing?

✓ Answer:

When resizing from 2^i to $2^{(i+1)}$:

- Need 2^i dollars to copy elements
- Those dollars were already saved from earlier inserts

So:

✓ No debt

✓ No extra cost

◆ PROMPT 14: Why does this guarantee correctness?

✓ Answer:

Because:

- Previous overflow happened at $2^{(i-1)}$

- Cells $2^{(i-1)}$ through $2^i - 1$ still have their saved dollars
- Those dollars were never spent

◆ PROMPT 15: Visual timeline of appends

Append costs

```
Append #:   1 2 3 4 5 6 7 8 9
Cost:       1 1 1 1 1 1 1 1 9
```

But **total cost** is still linear.

◆ PROMPT 16: Simple array growth illustration

Before resize

```
Index:   0 1 2 3 4 5 6 7
Array: [a b c d e f g h]
        $ $ $ $
```

Cells 4–7 each hold **2 saved dollars**.

Resize happens

```
New capacity = 16

Copy 8 elements ← paid by saved dollars

Insert new element

Store 2 dollars at new cell
```

◆ PROMPT 17: Why is total cost $O(n)$?

✓ Answer:

- Each element is copied **at most once per doubling**
- Number of doublings $\leq \log n$
- Total copied elements $\leq 2n$

Thus:

$$\text{Total cost} = O(n)$$

◆ PROMPT 18: What is the amortized cost per append?

✓ Answer:

$$\frac{O(n)}{n} = O(1)$$

So amortized append = constant time

◆ PROMPT 19: What does Proposition 5.1 state?

✓ Answer:

Performing **n append operations** on a dynamic array (starting empty) takes **$O(n)$ total time**

◆ PROMPT 20: Why is $\Omega(n)$ mentioned?

✓ Answer:

- A **single append** may take $\Omega(n)$
- But **amortized append** is $O(1)$
- Both statements are true simultaneously

◆ PROMPT 21: Key takeaway sentence (exam-ready)

✓ Answer:

Although a single append operation on a dynamic array may take $\Omega(n)$ time, a sequence of **n** append operations runs in **$O(n)$ total time** using amortized analysis.

◆ PROMPT 22: Real-world analogy 🏠

✓ Answer:

Like moving houses:

- Most days $\rightarrow$ no move
- Occasionally $\rightarrow$ huge move
- Average cost per day $\rightarrow$ small

◆ PROMPT 23: Where is this used in practice?

✓ Answer:

- Python lists
- Java ArrayList
- C++ vector
- JavaScript arrays

◆ PROMPT 24: Final mental model 🧠

Rare expensive operations

Many cheap operations

Saved work pays future cost

Average stays constant

Geometric Increase vs Arithmetic Increase in Dynamic Arrays

◆ PROMPT 1: What does “geometric increase in capacity” mean?

✓ Answer:

A geometric increase means:

Each time the array resizes, its new capacity is multiplied by a constant factor.

Examples:

- Doubling $\rightarrow \times 2$
- Increase by 25% $\rightarrow \times 1.25$
- Increase by 50% $\rightarrow \times 1.5$

Mathematically:

$new\ capacity = \alpha \times old\ capacity, \alpha > 1$

◆ PROMPT 2: Why is geometric growth important?

✓ **Answer:**

Because it guarantees:

- Few resize events
- Linear total copying cost
- $O(1)$ amortized append time

The key idea:

The extra space is always proportional to the current size.

◆ PROMPT 3: Is doubling ($\times 2$) the only valid strategy?

✓ **Answer:**

No ✗

Any geometric factor > 1 works, such as:

- 1.25
- 1.5
- 2

All give:

Amortized append time = $O(1)$ Amortized append time = $O(1)$

◆ PROMPT 4: Why does the textbook focus on doubling?

✓ **Answer:**

Because doubling:

- Minimizes number of resizes
- Simplifies the proof
- Uses fewer resize operations

But it may waste more memory.

◆ PROMPT 5: What is the trade-off when choosing the growth factor?

✓ Answer:

Growth Factor	Runtime	Memory Waste
Large ($\times 2$)	Fewer resizes	More unused space
Small ($\times 1.25$)	More resizes	Less wasted space

☞ Trade-off:

- Runtime efficiency vs memory efficiency

◆ PROMPT 6: Why does $\times 1.25$ still give $O(1)$ amortized time?

✓ Answer:

Because:

- Each resize still increases capacity **proportionally**
- Total amount of copying remains **linear in n**
- We just need to charge more cyber-dollars per append

Instead of **3 dollars**, maybe **5 or 6 dollars**.

◆ PROMPT 7: What is the key condition for $O(1)$ amortized time?

✓ Answer:

The additional space must be proportional to the current array size.

This is the **core principle**.

◆ PROMPT 8: What is an arithmetic increase in capacity?

✓ Answer:

An **arithmetic increase** means:

Each resize adds a fixed number of cells.

Examples:

- +1
- +2
- +1000

Mathematically:

$$\text{new capacity} = \text{old capacity} + c$$

◆ PROMPT 9: Why is arithmetic growth tempting?

✓ Answer:

Because it:

- Minimizes unused memory
- Feels “careful” with space

But... it destroys performance 😞

◆ PROMPT 10: What happens if we increase by +1 each time?

✓ Answer:

Every append causes:

- Resize
- Copy all existing elements

Cost sequence:

$$1 + 2 + 3 + \dots + n$$

Total time:

$$\Omega(n^2)$$

◆ PROMPT 11: What about +2 or +3 growth?

✓ **Answer:**

Slightly better, but still **quadratic**.

Why?

- Resizes still happen too frequently
- Copying cost accumulates badly

◆ PROMPT 12: Visualizing arithmetic growth (Figure 5.15 idea)

 **Increase by +2**

Capacity: $2 \rightarrow 4 \rightarrow 6 \rightarrow 8 \rightarrow 10 \rightarrow \dots$

Copy cost: $2 + 4 + 6 + 8 + \dots$

Total:

$$2(1 + 2 + 3 + \dots) = \Omega(n^2)$$

 **Increase by +3**

Capacity: $3 \rightarrow 6 \rightarrow 9 \rightarrow 12 \rightarrow \dots$

Copy cost: $3 + 6 + 9 + \dots$

Still:

$$\Omega(n^2)$$

◆ PROMPT 13: Why doesn't large fixed increment (e.g., +1000) help?

✓ **Answer:**

Because:

- For large n , +1000 is tiny

- Eventually, resizing happens very frequently

Asymptotically:

$$\text{Still } \Omega(n^2)$$

◆ PROMPT 14: State Proposition 5.2 in simple words

✓ **Answer:**

Using a fixed increment for resizing causes dynamic array appends to take quadratic time.

◆ PROMPT 15: Mathematical proof idea (intuition)

✓ **Answer:**

Let:

- Increment = c
- Number of resizes $\approx n / c$

Total copy cost:

$$c + 2c + 3c + \dots + mc$$

Which becomes:

$$\theta(n^2)$$

◆ PROMPT 16: Key difference between geometric and arithmetic growth

✓ **Answer:**

Feature	Geometric	Arithmetic
Resize frequency	Rare	Frequent
Total copy cost	$O(n)$	$\Omega(n^2)$
Amortized append	$O(1)$	$\Omega(n)$
Practical use	✓ Yes	✗ No

◆ PROMPT 17: Why is this lesson important?

✓ Answer:

Because:

- Small design choices → huge performance differences
- Correct asymptotic analysis guides good data structure design

Memory Usage and Shrinking a Dynamic Array

Below are **all the key learning prompts**, each followed by **clear explanations** and **simple drawings**.

◆ PROMPT 1: Why does geometric growth guarantee $O(n)$ memory usage?

✓ Answer:

When a dynamic array grows geometrically:

- Capacity is always proportional to the number of elements
- Capacity never explodes far beyond what is needed

If:

- n = number of elements
- C = array capacity

Then:

$$C = O(n)$$

This is ideal — memory grows **linearly**, not quadratically.

◆ PROMPT 2: Why is $O(n)$ memory usage desirable?

✓ Answer:

Because:

- It scales well for large data
- No excessive unused memory
- Predictable resource consumption

In short:

Efficient time **and** efficient space.

◆ **PROMPT 3: What new problem arises when we allow removals?**

✓ **Answer:**

If elements can be removed:

- The array may stay very large
- Even when few elements remain

Example:

1. Insert 1,000,000 elements $\rightarrow$ capacity $\approx$ 1,048,576
2. Remove 999,999 elements
3. Array still occupies huge memory 😞

Now:

capacity $\gg$ number of elements

◆ **PROMPT 4: Why can't we ignore this problem?**

✓ **Answer:**

Because:

- Memory is a limited resource
- Long-running programs can waste enormous space
- Violates the $O(n)$ memory guarantee

◆ **PROMPT 5: What is the naive shrinking strategy?**

✗ **Bad Idea:**

Shrink the array immediately whenever an element is removed.

Why this fails:

- Causes frequent grow/shrink cycles
- Breaks amortized $O(1)$ time

This is called **thrashing**.

◆ PROMPT 6: What is oscillation (thrashing)?

✓ **Answer:**

Oscillation happens when:

- One operation grows the array
- The next shrinks it
- And this repeats rapidly

Oscillation Example

Insert → grow

Remove → shrink

Insert → grow

Remove → shrink

Each step costs $\Theta(n) \rightarrow$ **disaster**

◆ PROMPT 7: What is the key design goal when shrinking?

✓ **Answer:**

We must ensure:

1. **$O(n)$ memory usage**
2. **$O(1)$ amortized time**
3. **No rapid oscillation**

All three must hold simultaneously.

◆ PROMPT 8: What is the standard safe shrinking rule?

✓ Answer:

Halve the capacity only when the number of elements falls below one-fourth of capacity.

Formally:

If $n < 4C$, then resize to $2C$

◆ PROMPT 9: Why one-fourth and not one-half?

✓ Answer:

Because it creates a **buffer zone** that prevents oscillation.

If we shrank at half:

- Insert one element → grow
- Remove one → shrink
- Repeat endlessly ✕

The $\frac{1}{4}$ rule introduces **hysteresis**.

◆ PROMPT 10: What is hysteresis (in simple words)?

✓ Answer:

Using different thresholds for growing and shrinking to avoid rapid flipping.

Grow when:

$$n = C$$

Shrink when:

$$n < 4C$$

◆ PROMPT 11: What memory bound does this guarantee?

✓ Answer:

After shrinking:

$$\frac{C}{2} \leq 2n$$

Before shrinking:

$$C \leq 4n$$

So overall:

$$n \leq C \leq 4n$$

☞ Capacity is **never more than 4× the number of elements**

◆ PROMPT 12: Why is this $O(n)$ memory usage?

✓ **Answer:**

Because:

$$C = \Theta(n)$$

Constant factor (4) does not matter asymptotically.

◆ PROMPT 13: Does shrinking break amortized $O(1)$ time?

✓ **Answer:**

No ✓

Because:

- Shrinking happens **rarely**
- Many deletions occur between shrinks
- Saved “cyber-dollars” pay for copying

Same amortized logic as growing.

◆ PROMPT 14: Visual example of safe shrinking

Example

Capacity = 16

Elements = 15 → no shrink

Elements = 8 → no shrink

Elements = 4 → no shrink

Elements = 3 → shrink to 8

After shrink:

Capacity = 8

Elements = 3

No immediate re-growth → stable

◆ PROMPT 15: Why not shrink earlier?

✓ Answer:

Shrinking earlier:

- Increases resize frequency
- Risks oscillation
- Breaks amortized guarantees

The $\frac{1}{4}$ rule is a **sweet spot**.

◆ PROMPT 16: How does this relate to Python lists?

✓ Answer:

Python's list implementation:

- Uses geometric resizing
- Uses delayed shrinking
- Avoids oscillation internally

That's why Python lists are fast and memory-efficient.

Python's List Class and Amortized Append

◆ PROMPT 1: How does Python store lists internally?

✓ Answer:

- Python lists are implemented using **dynamic arrays**.

- The array grows as elements are added.
- The exact resizing strategy is **neither purely geometric nor purely arithmetic**.
- Python uses a **custom growth pattern** to balance memory usage and speed.

◆ PROMPT 2: How can we measure the time complexity of append?

✓ Answer:

- Individual `append` operations are too fast to measure accurately.
- Instead, we measure the **average time over a series of n appends**.
- This gives us a good estimate of the **amortized cost**.

◆ PROMPT 3: How is the experiment implemented in Python?

✓ Answer:

From Code Fragment 5.4:

```
from time import time

def compute_average(n):
    """Perform n appends to an empty list and return average time elapsed."""
    data = []
    start = time()
    for k in range(n):
        data.append(None)
    end = time()
    return (end - start) / n # average time per append
```

- `time()` measures start and end.
- `for k in range(n)` appends `None` n times.
- Average time is $(\text{end} - \text{start}) / n$.

◆ PROMPT 4: What does “amortized constant-time” mean here?

✓ Answer:

- **Amortized constant time** means:

Each append costs $O(1)$ on average, even though some individual appends (resizing) may cost more.

- Most appends are **very fast**.
- Rare resizes are **expensive**.
- When averaged over all operations, **time per append is constant**.

◆ PROMPT 5: Why is the time higher for small n ?

✓ Answer:

- Loop overhead is significant for small lists.
- Measurement noise affects very short operations.
- The final resize event has a proportionally bigger impact when n is small.

◆ PROMPT 6: What do the experimental results show?

n	100	1,000	10,000	100,000	1,000,000	10,000,000	100,000,000
μs	0.219	0.158	0.164	0.151	0.147	0.147	0.149

Interpretation:

- As n increases, **average append time stabilizes** around 0.147–0.149 μs.
- Confirms **amortized $O(1)$ behavior**.

◆ PROMPT 7: How does Python avoid wasting too much memory?

✓ Answer:

- Python does not strictly double the array every time.
- Instead, it grows by a **variable factor** depending on the current size.
- This reduces memory waste but still guarantees **amortized constant time**.

◆ PROMPT 8: How is resizing handled?

✓ Answer:

- When the array is full:
 - Allocate a new array with a larger capacity.
 - Copy existing elements to the new array.
 - Append the new element.
- Rarely occurs, so **most appends are fast**.